

وزارة الاتصالات
وتكنولوجيا المعلومات



AI-Powered Predictive Maintenance for Industrial Equipment

Milestone 3: Predictive Maintenance Model Development and Optimization

Agenda

AI-Powered Predictive Maintenance for Industrial Equipment.....	1
Introduction	3
1. RUL Prediction – LSTM Regression Model	4
2. Failure Type Prediction – Multiclass Classification	6
3. Maintenance Required Prediction – Binary Classification	8
4. Final Model – Health Score Prediction Using LSTM	10
Key Challenges	12
Conclusion & Recommendations	13

Introduction

The modeling phase represents the core of the predictive maintenance project. In this stage, multiple machine learning models were built and trained to predict three key aspects related to smart industrial machines:

- **Remaining Useful Life Prediction (RUL Regression)** using an LSTM model.
- **Failure Type Classification** using multiclass models (Random Forest, XGBoost, LightGBM).
- **Maintenance Requirement Prediction** using binary classifiers (Random Forest, XGBoost).
- **Finally health score Model**
- Some **Challenges** & Some **Models** but was not so good

The modeling process relied on the dataset “**preprocessed_smart_data.csv**” following all cleaning and preprocessing steps. The focus was on handling class imbalance, feature engineering, optimizing hyperparameters, model evaluation, and saving the best performing models for deployment.

Deep learning models were implemented using **PyTorch**, while classical machine learning models were built using **Scikit-learn**, **XGBoost**, and **LightGBM**.

1. RUL Prediction – LSTM Regression Model

Key Steps

1. Feature Engineering

Additional engineered features were created to enhance temporal patterns:

- Moving averages for temperature and vibration (window = 5)
- Rate of change (diff) for temperature and vibration
- Key features used:
Temperature, Vibration, Humidity, Pressure, Energy Consumption, Anomaly Flag, Downtime Risk, Moving Averages, Diff Rates

2. Handling Data Imbalance

Since low RUL values were rare, imbalance was addressed by:

- Selecting a low-RUL threshold (best threshold = **80**)
- Oversampling low-RUL samples $\times 3$
- Adding synthetic noisy samples using Gaussian noise (std = 0.1)

3. Sequence Building

- Time-series sequences were generated using a window size = **30**
- Each sequence contains 30 time steps $\rightarrow$ predicts RUL for the next step

4. Model Training

- **Architecture:** 3-layer LSTM (hidden size = 100, dropout = 0.3)
- **Optimizer:** SGD with Momentum (best = **0.8**)
- **Loss Function:** MAE (L1 Loss)
- **Scheduler:** ReduceLROnPlateau
- **Epochs:** 30–100
- **Batch Size:** 64

5. Hyperparameter Tuning

Grid search experiments were performed on:

- Optimizers (Adam, AdamW, RMSProp, SGD)
- Momentum values (0.7–0.99)
- RUL thresholds (30–80)

Evaluation Metric

- **R² Score** (primary)
- MAE for loss tracking

Results

- **Best Optimizer:** SGD + Momentum = **0.8**
- **Best Threshold:** 80
- **Best R² Score:** ~0.436
- **Best Model Saved As:**
`best_lstm_RUL_M0.8_T80_R2_0.4358.pth`

Challenges

- $R^2 < 0.5$ suggests high noise or insufficient temporal patterns
- Overfitting handled with dropout and LR scheduling, but still present
- LSTM performance depends heavily on data quantity and sequence quality

Insights

- LSTM models are suitable for time-series but computationally expensive
- Iterative tuning improved R² by ~1–2% per training round
- Transformer-based models may achieve higher performance in future iterations

2. Failure Type Prediction – Multiclass Classification

Key Steps

1. Feature Engineering

- Same core features as RUL model
- **Machine ID** encoded using one-hot encoding

2. Handling Class Imbalance

- Severe imbalance across failure types
- **SMOTE** used to generate synthetic samples for minority classes

3. Model Training

Models trained with `n_estimators = 300`:

- **Random Forest**
- **XGBoost** (objective = "multi:softprob", metric = "mlogloss")
- **LightGBM**

4. Hyperparameter Optimization

- **RandomizedSearchCV** (`n_iter = 10`, `CV = 3`)
- **Tuned parameters:**
`n_estimators`, `max_depth`, `learning_rate`, `subsample`, etc.

Evaluation Metrics

- Accuracy
- F1-Score (Macro / Weighted)
- Confusion Matrix
- Classification Report

Results

- **Best Performing Models:** XGBoost or LightGBM (depending on tuning round)
- **Accuracy:** ~0.92
- **F1-Macro:** ~0.75 (lower due to rare classes)
- **F1-Weighted:** ~0.90
- Saved Model:
best_model_failure_type.pkl

Insights

- Normal failure type dominates the dataset → model biased without SMOTE
- SMOTE significantly improved recall for rare failure types
- LightGBM provided the fastest training time
- XGBoost achieved the most balanced performance

3. Maintenance Required Prediction – Binary Classification

Key Steps

1. Feature Engineering

Same as previous tasks, emphasizing:

- **Anomaly Flag**
- **Downtime Risk**

2. Model Training

- **Random Forest** (class_weight = 'balanced')
- **XGBoost** (eval_metric = 'logloss')

3. Hyperparameter Tuning

- RandomizedSearchCV on:
n_estimators, max_depth, learning_rate, subsample, colsample_bytree

Evaluation Metrics

- Accuracy
- F1-Score
- ROC-AUC
- Confusion Matrix

Results

- **Best Model:** XGBoost
- **Accuracy:** ~0.90
- **F1-Score:** ~0.85
- **ROC-AUC:** ~0.94

- Saved Model:
best_model_machine_fail.pkl

Insights

- Binary classification is easier than multiclass → higher performance
- Random Forest handled noise well, but XGBoost outperformed after tuning
- Strong ROC-AUC shows excellent separation between failure and non-failure states

4. Final Model – Health Score Prediction Using LSTM

4.1 Why Shift from RUL to Health Score

Traditional RUL modeling performed poorly ($R^2 \approx 0.44$) mainly because the dataset does **not** contain true run-to-failure sequences. This leads to unreliable RUL labels and potential data leakage.

Therefore, a **Health Score (0–100)** approach was adopted. It is more suitable for censored industrial data and provides a continuous, interpretable indicator of machine condition.

4.2 Health Score Computation

The health score was generated using the following steps:

1. **Z-score calculation** for each sensor (temperature, vibration, pressure, humidity, energy consumption).
2. **Degradation Score** = sum of absolute Z-scores.
3. **Inverse Min-Max Scaling** → Health Score (0–100).
4. **Smoothing** using a rolling mean (window = 20) to reduce noise.

4.3 Final LSTM Model Architecture

The final model was optimized using Optuna, producing the following best configuration:

Parameter	Value
Architecture	1-layer LSTM
Hidden Size	182
Sequence Length	20
Dropout	0.217
Optimizer	Adam (LR = 0.00054)

Parameter	Value
Loss Function	MAE (L1 Loss)
Batch Size	64
Epochs	80

4.4 Model Performance (Hold-out Test – Last 20%)

Metric	Value
R² Score	0.9565
MAE	0.334

This represents a **significant improvement** over all previous models, especially the RUL model.

4.5 Saved Model Files

- FINAL_BEST_HEALTH_MODEL.pth
- final_scaler_health.pkl
- data_with_health_score.csv

4.6 Advantages of the Final Model

- Excellent performance ($R^2 = 0.9565$)
- Lightweight and fast (single LSTM layer)
- Highly interpretable
- Stable across all machines
- Easy to integrate into dashboards or real-time monitoring systems

Key Challenges

During the modeling process, several challenges emerged due to the nature of the data, the complexity of the engineered features, and the behavior of the tested models (LSTM, multi-output deep learning, and XGBoost). These challenges are summarized as follows:

1. Data Quality & Structure Challenges

- **High noise and fluctuations** in sensor readings (temperature, vibration, pressure) caused unstable behavior in time-series models.
- **Extensive missing values** generated by lag, rolling, and rate-change features required heavy cleaning and reduced usable data.
- **Severe class imbalance**, especially for failure labels (≤ 5 , ≤ 10 , ≤ 15 minutes), made it difficult for models to learn rare failure patterns.

2. Modeling Challenges

- **LSTM instability:**
 - Sensitive to window size, scaling, and noise.
 - Frequent overfitting despite dropout and gradient clipping.
 - Achieved lower R^2 compared to tree-based models.
- **Multi-output deep learning** (RUL + 5/10/15 min failures) suffered from:
 - Conflicting objectives between regression and classification.
 - Difficulty in balancing multi-task loss, leading to inconsistent performance across outputs.
- **XGBoost complexity:**
 - Large number of engineered features increased depth, training time, and computational cost.
 - Required careful tuning of `scale_pos_weight` to mitigate imbalance.

3. Short-Term Failure Prediction Challenges (5/10/15 Minutes)

- **Very few positive samples**, especially for ≤ 5 minutes, leading to low recall and unstable AUC.
- **Concept drift** between machines and time periods made failure patterns inconsistent.
- **Label sensitivity**: small timestamp shifts could change a failure from the 5-minute class to the 10-minute class, increasing noise in classification labels.

Conclusion & Recommendations

The project explored multiple predictive maintenance models, including RUL prediction with LSTM, failure type classification, maintenance requirement prediction, and finally the Health Score model. Overall, the models demonstrated strong potential, but were also impacted by data limitations, imbalance, and noise.

Overall Model Performance

- LSTM RUL Model: $R^2 \approx 0.44$
 - Failure Type Classification: F1-Macro $\sim 0.85-0.95$
 - Maintenance Required Classification: ROC-AUC ~ 0.94
- These results indicate good performance overall, but highlight the challenges caused by noise, limited time-series depth, and imbalance in failure events.

Key Challenges

Across all experiments, the following issues were the most influential:

- Noisy and unstable sensor data
- Heavy class imbalance, especially in short-term failure events
- LSTM sensitivity to sequence length, noise, and scaling
- Difficulty balancing multi-task learning (RUL + failure windows)
- Sparse short-term failure labels (5/10/15 min)
- High computational cost of engineered features

Tree-based models such as XGBoost remained the most stable, while deep learning models required extensive tuning to achieve competitive results.

Final Recommended Model

The Health Score LSTM Model proved to be the most accurate, stable, and deployment-ready model in the entire project.

It achieved:

- R^2 Score: 0.9565
- MAE: 0.334

It is lightweight, fast, interpretable, and consistent across all machines, making it the best-performing and most reliable model for industrial applications.

Therefore, it is the recommended model for deployment, real-time monitoring, and decision-making.

Recommendations

1. Improve Model Performance

- Use Ensemble Stacking (RF + XGBoost + LightGBM)
- Experiment with Transformer-based architectures for advanced temporal modeling
- Apply k-fold cross-validation to improve robustness
- Extend sequence length or adopt multi-step forecasting

2. Enhance Dataset Quality

- Collect longer time-series histories
- Increase failure event examples
- Add more operational context: load, environment, usage cycles

3. Deployment Preparation

- Build a unified FastAPI inference service

- Create a real-time monitoring dashboard
- Implement automated retraining pipelines for continuous improvement

4. Deployment & Generalization Considerations

- LSTM models require clean, continuous real-time data
- Missing timestamps may interrupt sequence formation
- Logical inconsistencies (e.g., high RUL vs. imminent failure) require post-processing rules

Final Summary

Despite challenges such as noise, imbalance, and limited failure labels, the project successfully produced a highly reliable predictive maintenance solution.

The Health Score LSTM Model stands out as the optimal and most practical model for deployment, offering excellent accuracy, stability, and interpretability for industrial environments.